Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 3

Requirements Engineering and Team Workflows

Version Control with Git Review

Professional Workflow Summary

- Branch per feature
- Pull requests
- Reviews before merge

Version Control with Git Review

Concept-to-Command Map

Concept	Command
Select Work for Save	git add
Save Work	git commit
Share Work	git push
Sync	git pull
Isolate Work	git branch
Propose Merge	Pull Request (GitHub)

What I Expect You To Do in Class Projects

- Never Work directly on main
- Commit early and often
- Write clear, thoughtful commit messages
- Use Pull Requests

The Git Mental Model

- **Local Work:**
  Branch → Edit → Add → Commit
- **Team Process:**
  Push → PR → Review → Merge

Why Requirements Matter

- Building the wrong thing is expensive
- Fixing mistakes later costs more
- Requirements align teams
- Clear requirements reduce rework

What Goes Wrong with Poor Requirements

- Ambiguity: Happens when requirements are vague or subjective
- Assumptions: Unspoken expectations
- Missing stakeholders: Decisions made without input from those who will use, support, or maintain the system

Functional vs. Non-Functional Requirements

- Functional Requirements
  - What the system does
  - Features and behaviors
- Non-Functional Requirements
  - How the system behaves
  - Quality attributes

Stakeholders and Perspectives

- Users
- Customers
- Developers
- Operators

Requirements Don’t Appear Fully Formed

- Stakeholders describe problems, not specifications
- Initial requests are often incomplete
- Engineers refine through questioning

Example: Stakeholder Statement

Stakeholder says: “Students need reminders for assignments.”

What questions should we ask?

User Stories

- Lightweight requirement format
- Focus on user value
- Structure:
  - As a …
  - I want …
  - So that …

Refining Into a User Story

- Weak: As a student, I want reminders so that I don’t forget assignments.
- Stronger: As a student, I want to receive a reminder 24 hours before an assignment is due so that I can prepare in advance.

The Engineer's Role

Engineers help define:

- Scope
- Constraints
- Tradeoffs
- Success criteria

The Engineer's Role

Not All User Stories Are Equal

- Following the template does not guarantee a Good User Story
- ”As a… I want… so that…” is necessary, not sufficient.
- Strong User Stories are
  - Specific
  - Focused
  - Testable
  - Valuable

Weak vs Strong User Story Example

- Weak: As a student, I want to manage assignments so that I stay organized.
- Stronger: As a student, I want to edit the due date of a personal task so that I can track non-course work.

Characteristics of Strong Stories

- Clear actor
- Specific action
- Clear value
- Narrow scope
- Independently testable

Acceptance Criteria

- Defines “Done”
- Makes Requirements Testable
- Reduces Ambiguity

Why Non-Functional Requirements Matter

Two systems can have identical features but radically different quality.

Non-functional requirements define quality.

Categories of Non-Functional Requirements

- Performance
- Scalability
- Reliability
- Security
- Usability
- Accessibility
- Maintainability
- Observability

Make Them Measurable

Weak:

The system should be fast.

Better:

Dashboard loads within 2 seconds under 500 concurrent users (95th percentile).

Tradeoffs and Constraints

Improving one quality attribute often affects others.
Examples:

- Performance vs cost
- Security vs usability
- Speed of delivery vs maintainability

Quick Exercise

Rewrite this non-functional requirement:

"The system should be secure."

Requirements as Living Artifacts

- Requirements evolve
- Iterative refinement
- Change is expected

Requirements Document Example: Project Context

- Project: Student Assignment Tracker
- Audience: University Students
- Goal: Reduce missed deadlines and planning friction

Requirements Document Example: Initial Draft (v0.1)

- The system should be user friendly
- Students can manage assignments
- Assignments should update automatically
- The system should work on all devices

Requirements Document Example: Problems with Initial Draft

- Vague and subjective language
- Unclear scope of “Manage”
- No definition of “automatically”
- No success or completion criteria

Requirements Document Example: Revised Requirements (v1.0)

- Focus on student planning and visibility
- Explicit definition of system behavior
- Clear boundaries of functionality

Requirements Document Example: Revised User Stories

- As a student, I want to view all upcoming assignments so that I can plan my week.
- As a student, I want assignments grouped by course so that I can balance my workload.
- As a student, I want overdue assignments highlighted so that I can take corrective action.

Requirements Document Example: Acceptance Criteria

- Each assignment displays course, title, and due date
- Assignments are sorted by due date within each course
- Overdue assignments are visually highlighted
- Empty state shown if no assignments exist

Requirements Document Example: Non-Functional Requirements

- Assignment list loads within 2 seconds on typical campus Wi-Fi
- Interface supports keyboard-only navigation
- System supports at least 500 concurrent users

Requirements as Code (Git Workflow)

- Requirements stored as markdown in requirements.md
- Changes proposed via pull requests
- Peer review focuses on clarity and risk

Takeaways

- Clear requirements reduce rework
- User Stories define system behavior
- Acceptance criteria define "done“
- Non-functional requirements define “how”
- Git supports collaborative understanding

Collaborative Requirements in Git

- Markdown files
- Issues for discussion
- Pull requests for changes

Pull Requests for Requirements

- Review and discussion
- Comments and feedback
- Approval workflows

Resolving Conflicts

- Technical conflicts
- Human communication issues

Next Steps

- Draft initial user stories
- Think about acceptance criteria
- Prepare for Git-based collaboration

Assignment: Weekly Standup Reflection

https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection

Due Feb 19 (and every week thereafter)

***For Feb 19 only, don't worry about points 5 or 6 under "What to Do", as we have not yet covered those in class.**

Assignment: Requirements Report Document

https://rrostock1.github.io/Ursinus-CS375/Project/Requirements

Due Feb 26